

Assignment 4

STAT 8670

Tommy Stell
Georgia State University

05 November, 2025

Contents

1	Question 1	2
1.1	1a	2
1.2	1b	2
2	Question 2	3
2.1	2a	3
2.2	2b	4
2.3	2c	5
3	Question 3	7
3.1	3a	7
3.2	3b	8
4	Question 4	9
4.1	4a	9
4.2	4b	10
5	Question 5	11
5.1	5a	11
5.2	5b	12
6	Question 6	14
6.1	6a	14

1 Question 1

1.1 1a

Explain the difference between

a.) Training-test split b.) K-fold cross-validation c.) Leave one out cross validation

□

a.) Training-test split

This is a random split of the original dataset into two disjoint datasets. This is done to allow your data set to have both something to form the model with (i.e. the training set) and further a set to score the accuracy of your trained data against (testing set). This is done so that the labeled data can be used to test the accuracy of the model without tainting the testing phase by including the data used a testing metric in the development of the model.

b.) K-fold cross-validation

This splits the dataset into K equal disjoint subsets of data. It then selects a single fold (for example, the i th fold) and trains the model on the remaining folds. It then uses the i th fold left out initially to evaluate the model. This process then cycles through so each one of the K folds is used to evaluate the training on the remaining folds.

c.) Leave one out cross validation (LOOCV)

This is a variation of K-fold CV where $K = n$ so each fold contains a single data point. The model is then used to train on the other $n - 1$ data points and evaluated on the data point left out.

The bias-variance trade off:

a.) Training-test split

Having a large fraction of the data sequestered into the test set gives a low variance on the trade off of a high bias. This is due to a significant decrease in the size of the training set. Conversely, if the test set is a small fraction of the total dataset, then the test error will be noisy and less meaningful.

b.) K-fold cross-validation

Low $\$K\$$ values lead to high bias and low variance while higher $\$K\$$ values lead to low bias and higher variance.

c.) Leave one out cross validation (LOOCV)

Low bias since it uses practically the entire dataset for training but variance is much higher due to the sensitivity of the estimate to the single point left out.

■

1.2 1b

Why is cross-validation often preferred over analytical MSE formulas for model-selection?

□

First, this assumes that an analytical MSE formula can be derived. A lot of models I deal with may not have a specific analytical formula for the MSE. This leads to CV being preferred for cases where it may be either too labor intensive or impossible to determine an analytical formula.

Further, CV actually simulates the data for a variety of related models. This can lead to stronger models just due to the usage of an empirical estimation of the model.

■

2 Question 2

Consider the model

$$Y = \beta_0 + \beta_1 x + \beta_2 x^2 + \epsilon, \quad \epsilon \sim N(0, \sigma^2)$$

With $x_i \sim \text{Unif}(-2, 2)$ and $\beta_0 = 1, \beta_1 = 2, \beta_2 = -1$.

2.1 2a

Generate $n = 60$ observations, fit polynomial models of degree $d = 1, \dots, 6$ and perform a 5-fold CV. Compute mean CV error for each d .

□

```
Q2_model <- function(n, beta0 = 1, beta1 = 2, beta2 = -1, sigma = 1) {
  X1 <- runif(n, min = -2, max = 2)
  epsilon <- rnorm(n, mean = 0, sd = sigma)
  Y <- beta0 + beta1 * X1 + beta2 * (X1^2) + epsilon
  data.frame(X1 = X1, Y = Y)
}

# question 2.a

Q2_observations <- Q2_model(60)

Q2_poly_degree <- c(1, 2, 3, 4, 5, 6)

Q2_CV <- function(data, degrees, k = 5) {
  n <- nrow(data)
  folds <- sample(rep(1:k, length.out = n))
  cv_errors <- numeric(length(degrees))

  for (d in seq_along(degrees)) {
    degree <- degrees[d]
    fold_errors <- numeric(k)

    for (fold in 1:k) {
```

```

train_data <- data[folds != fold, ]
test_data <- data[folds == fold, ]

model <- lm(Y ~ poly(X1, degree), data = train_data)
predictions <- predict(model, newdata = test_data)
fold_errors[fold] <- mean((test_data$Y - predictions)^2)
}

cv_errors[d] <- mean(fold_errors)
}

names(cv_errors) <- paste0("Degree_", degrees)
return(cv_errors)
}

print("MSE for polynomials of degree 1,...,6")

```

```
## [1] "MSE for polynomials of degree 1,...,6"
```

```
Q2_CV(Q2_observations, Q2_poly_degree)
```

```
## Degree_1 Degree_2 Degree_3 Degree_4 Degree_5 Degree_6
## 2.507595 1.251584 1.256372 1.209549 1.247069 1.286102
```



2.2 2b

Plot CV error vs degree d and identify the optimal model.



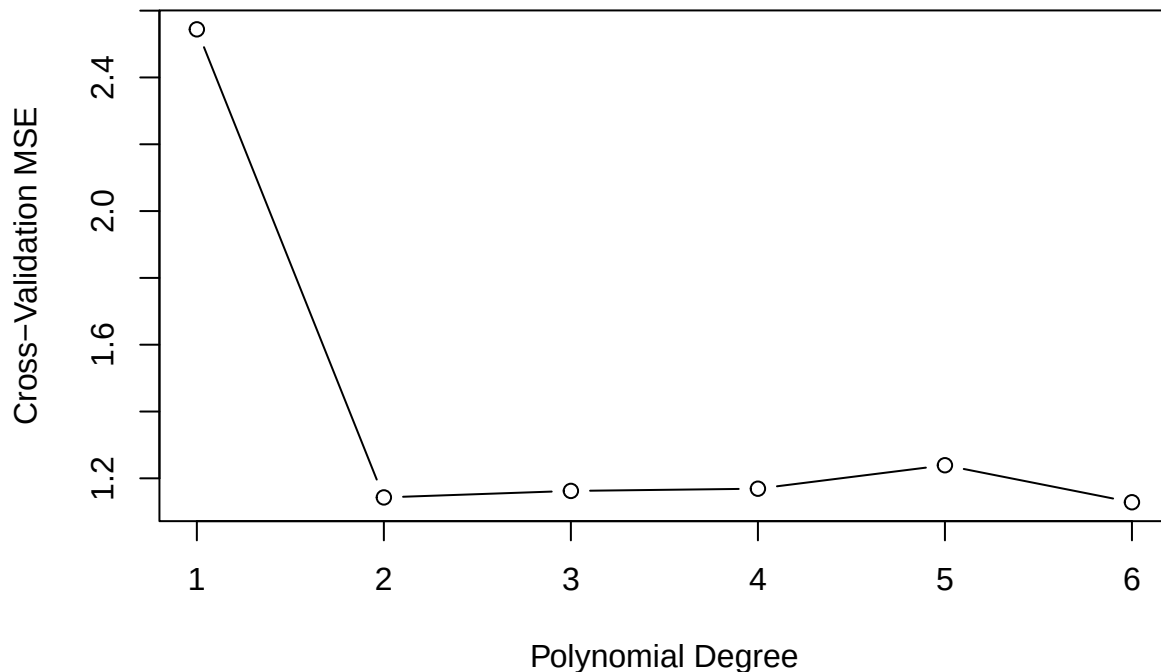
```

Q2_plot_CV_errors <- function(cv_errors) {
  plot(as.numeric(sub("Degree_", "", names(cv_errors))), cv_errors, type = "b",
       xlab = "Polynomial Degree", ylab = "Cross-Validation MSE",
       main = "CV MSE vs Polynomial Degree")
}

Q2_plot_CV_errors(Q2_CV(Q2_observations, Q2_poly_degree))

```

CV MSE vs Polynomial Degree



From the plot, we can see that the MSE drops and approximately plateaus after degree 2 which means to avoid underfitting, we should have a degree at least 2. To avoid overfitting however, we want the minimum degree polynomial so we want the polynomial with the first degree after the “elbow”. So the optimal model is of degree 2.



2.3 2c

Compare your result with training MSE only (no CV). Explain the difference.



```
Q2_train_test_split <- function(data, train_frac = 0.7) {  
  n <- nrow(data)  
  train_indices <- sample(1:n, size = floor(train_frac * n))  
  train_data <- data[train_indices, ]  
  test_data <- data[-train_indices, ]  
  list(train = train_data, test = test_data)  
}  
  
Q2_MSE <- function(train_data, test_data, degree) {  
  model <- lm(Y ~ poly(X1, degree), data = train_data)  
  predictions <- predict(model, newdata = test_data)
```

```

    mean((test_data$Y - predictions)^2)
}

Q2_main <- function() {
  data <- Q2_model(60)

  cv_errors <- Q2_CV(data, Q2_poly_degree)

  #Q2_plot_CV_errors(cv_errors)

  splits <- Q2_train_test_split(data)
  train_data <- splits$train
  test_data <- splits$test

  test_MSEs <- sapply(Q2_poly_degree, function(d) {
    Q2_MSE(train_data, test_data, d)
  })

  print("CV error of different degrees")
  names(cv_errors) <- paste0("Degree_", Q2_poly_degree)
  print(cv_errors)
  print("")
  print("MSE of different degrees")
  names(test_MSEs) <- paste0("Degree_", Q2_poly_degree)
  print(test_MSEs)
}

Q2_main()

```

```

## [1] "CV error of different degrees"
## Degree_1 Degree_2 Degree_3 Degree_4 Degree_5 Degree_6
## 2.495536 1.248149 1.143174 1.236721 1.355242 1.353804
## [1] ""
## [1] "MSE of different degrees"
## Degree_1 Degree_2 Degree_3 Degree_4 Degree_5 Degree_6
## 2.843238 1.297125 1.209061 1.340342 1.345632 1.363549

```

Overall, the MSE is lower than the CV error of the different degrees. Both methods of calculating errors pick out a order 2-3 polynomial. Curiously, the MSE of a degree 3 model is lower than the MSE of a degree 2 model which is odd due to the true nature of the model being a degree 2 polynomial. Knowing this, the CV is more reliable because it allows us to pick a model which fits the true nature of the data better. The MSE alone fails to distinguish between the correct model and one that is overfit.

■

3 Question 3

For linear regression $\hat{Y} = HY$, where $H = X(X^T X)^{-1} X^T$.

3.1 3a

Show that LOOCV prediction errors can be computed without refitting

$$\text{err}_{\text{LOOCV}} = \frac{1}{n} \sum_{i=1}^n \left(\frac{Y_i - \hat{Y}_{(i),i}}{1 - h_{ii}} \right)^2$$

Where h_{ii} is the i th leverage.

□

For simplicity, let us denote the following, $X \in \mathbb{R}^{n \times p}$ is the dataset with n data points and p parameters, $X_{(i)}$ is the dataset without the i th observation, $H_{(i)}$ is the constructed H matrix without the i th component. Further, Y_i is the observed value at the i th data point, $\hat{Y}_i$ is the fitted value at the i th datapoint. $Y_{(i)}$ is the all the observations without Y_i . $Y_{(i),i}$ is the predicted value for the i th datapoint fitted without seeing the i th datapoint.

We can express $\hat{Y}_i$ as follows.

$$\hat{Y}_i = HY = h_{ii}Y_i + \sum_{j \neq i} h_{ij}Y_j$$

This means we can write $\hat{Y}_{(i),i}$ as $\hat{Y}_i - h_{ii}Y_i$. This is simply the fitted prediction without the i th component. We can then take the difference between the preceding terms to get the following.

$$\begin{aligned} \hat{Y}_i - \hat{Y}_{(i),i} &= h_{ii}(Y_i - \hat{Y}_{(i),i}) \\ \implies Y_i - \hat{Y}_{(i),i} &= \frac{Y_i - \hat{Y}_i}{1 - h_{ii}} \end{aligned}$$

The above can then be used to determine the MSE between the true observation at the i th datapoint and the LOO predicted value for every possible datapoint we can remove. This results in the following.

$$\text{err}_{\text{LOOCV}} = \frac{1}{n} \sum_{i=1}^n (Y_i - Y_{(i),i})^2 = \frac{1}{n} \sum_{i=1}^n \left(\frac{Y_i - \hat{Y}_i}{1 - h_{ii}} \right)^2$$

So we can find the LOOCV error without refitting.

■

3.2 3b

Simulate $n = 40, p = 3$ features with noise $\sigma = 1$. Compare LOOCV MSE computer via refitting and via the shortcut formula. Confirm equality up to rounding error.

□

```
Q3_n <- 40
Q3_p <- 3
Q3_sigma <- 1

Q3_X <- matrix(rnorm(Q3_n * Q3_p), nrow = Q3_n, ncol = Q3_p)
Q3_data <- data.frame(y = rnorm(Q3_n, sd = Q3_sigma), Q3_X)

Q3_fit_model <- function(data) {
  model <- lm(y ~ ., data = data)
  summary(model)
}

Q3_X_matrix <- model.matrix(lm(y ~ ., data = Q3_data))

Q3_H <- Q3_X_matrix %*% solve(t(Q3_X_matrix) %*% Q3_X_matrix) %*% t(Q3_X_matrix)

h <- diag(Q3_H)

Q3_shortcut_LOOCV <- function(data, h) {
  n <- nrow(data)
  model <- lm(y ~ ., data = data)
  residuals <- resid(model)
  loo_cv_errors <- (residuals / (1 - h))^2
  mean(loo_cv_errors)
}

Q3_refitting_LOOCV <- function(data) {
  n <- nrow(data)
  loo_cv_errors <- numeric(n)

  for (i in 1:n) {
    train_data <- data[-i, ]
    test_data <- data[i, , drop = FALSE]

    model <- lm(y ~ ., data = train_data)
    prediction <- predict(model, newdata = test_data)

    loo_cv_errors[i] <- (test_data$y - prediction)^2
  }

  mean(loo_cv_errors)
```



```

}

Q3_main <- function() {
  fit_summary <- Q3_fit_model(Q3_data)

  time_shortcut_start <- Sys.time()
  loocv_shortcut_mse <- Q3_shortcut_LOOCV(Q3_data, h)
  time_shortcut_end <- Sys.time()
  time_refit_start <- Sys.time()
  loocv_refit_mse <- Q3_refitting_LOOCV(Q3_data)
  time_refit_end <- Sys.time()

  absolute_difference <- abs(loocv_shortcut_mse - loocv_refit_mse)

  print(paste("LOOCV MSE (Shortcut):", loocv_shortcut_mse))
  print(paste("LOOCV MSE (Refitting):", loocv_refit_mse))
  print(paste("Absolute Difference:", absolute_difference))
  print(paste("Time taken (Shortcut):", time_shortcut_end - time_shortcut_start))
  print(paste("Time taken (Refitting):", time_refit_end - time_refit_start))
}

Q3_main()

```

```

## [1] "LOOCV MSE (Shortcut): 0.78136713576329"
## [1] "LOOCV MSE (Refitting): 0.78136713576329"
## [1] "Absolute Difference: 0"
## [1] "Time taken (Shortcut): 0.000905990600585938"
## [1] "Time taken (Refitting): 0.0603828430175781"

```

We can see that the difference between the refitting and shortcut methods are the same within machine precision. The main difference is that the time for computing the MSE via the refitting was approximately two orders of magnitude larger than the time it took for the shortcut method of finding the MSE. This points towards the shortcut method being preferred for time complexity reasons.

■

4 Question 4

Let $X_1, \dots, X_n \sim N(\mu, 1)$ with $\mu = 0$. Estimate the standard error of the sample median $\hat{\theta}$.

4.1 4a

Write the non-parametric bootstrap algorithm for estimating standard error (SE).

□

Algorithm:

Step 1:

Compute $\hat{\theta} = \text{median}\{X_1, \dots, X_n\}$

Step 2:

For $1 \leq i \leq B$: Sample in $X = \{X_1, \dots, X_n\}$, denote $X_{(i)}$ as the i th bootstrapped sample. Compute $\hat{\theta}_{(i)} = \text{median}(X_{(i)})$. Collect all $\hat{\theta}_{(i)}$

Step 3:

Compute the bootstrap standard error shown below.

$$MSE_{(bootstrap)} = \sqrt{\frac{1}{B-1} \sum_{i=1}^B (\hat{\theta} - \hat{\theta}_{(i)})^2}$$

■

4.2 4b

Simulate $n = 50, B = 2000$. Compare bootstrap SE with theoretical asymptotic $SE \approx 1/(2f(\mu)\sqrt{n})$ where $f(\mu) = 1/\sqrt{2\pi}$.

□

```
Q4_sample_median_error <- function(n = 50, B = 2000) {  
  medians <- numeric(B)  
  
  for (b in 1:B) {  
    sample_data <- rnorm(n)  
    medians[b] <- median(sample_data)  
  }  
  
  median_estimate <- mean(medians)  
  temp <- mean((medians - median_estimate)^2)  
  
  MSE_bootstrap <- ((1/(B-1)) * sum((medians - median_estimate)^2))^0.5  
  
  list(median_estimate = median_estimate, mse = MSE_bootstrap)  
}  
  
Q4_compute_theoretical <- function(n = 50, mu = 0, sigma = 1) {  
  f_mu <- 1 / (2*pi)^0.5  
  theoretical <- 1/(2*(n^0.5)*f_mu)  
  return(theoretical)  
}
```

```

Q4_compare_errors <- function() {
  n <- 50
  B <- 2000

  sim_results <- Q4_sample_median_error(n, B)
  theoretical_mse <- Q4_compute_theoretical(n)

  print(paste("Simulated Median Estimate:", sim_results$median_estimate))
  print(paste("Simulated MSE:", sim_results$mse))
  print(paste("Theoretical MSE:", theoretical_mse))
  print(paste("MSE error:", abs(theoretical_mse - sim_results$mse)))
}

Q4_compare_errors()

```

```

## [1] "Simulated Median Estimate: 0.00161057056032594"
## [1] "Simulated MSE: 0.171451203511472"
## [1] "Theoretical MSE: 0.177245385090552"
## [1] "MSE error: 0.00579418157907935"

```

The bootstrap MSE is very close to the theoretical MSE. This points towards the bootstrap MSE being a good method to determining the $\hat{\theta}$.

■

5 Question 5

Let $\hat{\theta} = \max(X_1, \dots, X_n)$ for $X_i \sim \text{Unif}(0, 1)$.

5.1 5a

Derive jackknife bias and variance estimators in terms of leave one out $\hat{\theta}_{(i)}$.

□

We have that $\hat{\theta} = \max(X_1, \dots, X_n)$ for $X_i \sim \text{Unif}(0, 1)$. Define the LOO estimate as follows.

$$\hat{\theta}_{(i)} = \max(X_1, \dots, X_{i-1}, X_{i+1}, \dots, X_n)$$

And the entire jackknife estimate as:

$$\overline{\theta_{(\cdot)}} = \frac{1}{n} \sum_{i=1}^n \hat{\theta}_{(i)}$$

Since $\hat{\theta}$ is a measure of the maximum of the dataset, there are two possible cases for the ordered LOO estimates. Take X_j as the true maximum value of X . The two cases are thus either X_j is the unique maximum value or there exists X_k such that $X_j = X_k$.

In the case that X_j is the unique maximum, then the jackknife estimator $\hat{\theta}_{(j)}$ is less than the estimate of all other jackknife samples. Without loss of generality, assume that this is the $k - th$ sample or that $\hat{\theta}_{(j)} = X_k$. For every other estimate, $\hat{\theta}_{(i \neq j)} = X_j$. The expected value of $\hat{\theta}_{(i)}$ is thus as follows.

$$\bar{\hat{\theta}}_{(\cdot)} = \frac{(n-1)X_j + X_k}{n} = X_j - \frac{X_j - X_k}{n}$$

The bias is thus as follows.

$$\begin{aligned}\widehat{\text{Bias}}_{\text{jack}} &= (n-1)(\bar{\hat{\theta}}_{(\cdot)} - \hat{\theta}) = (n-1) \left(X_j - \frac{X_j - X_k}{n} - X_j \right) \\ \widehat{\text{Bias}}_{\text{jack}} &= \left(\frac{1-n}{n} \right) (X_j - X_k)\end{aligned}$$

The jackknife bias corrected estimator is thus $\hat{\theta}_{\text{jack}} = \hat{\theta} - \widehat{\text{Bias}}_{\text{jack}}$. This simplifies into the following.

$$\hat{\theta}_{\text{jack}} = X_j + \frac{n-1}{n}(X_j - X_k)$$

The variance can be computed in the standard manner by using the following equation.

$$\widehat{\text{Var}}_{\text{jack}} = \frac{n-1}{n} \sum_{i=1}^n (\hat{\theta}_{(i)} - \bar{\hat{\theta}}_{(\cdot)})^2$$

Using similar logic as with the bias, we get the following.

$$\begin{aligned}\widehat{\text{Var}}_{\text{jack}} &= \frac{n-1}{n} \left(\frac{(n-1)}{n^2} (X_j - X_k)^2 + \left(\frac{(n-1)(X_j - X_k)}{n} \right)^2 \right) \\ \widehat{\text{Var}}_{\text{jack}} &= \left(\frac{n-1}{n} \right)^2 (X_j - X_k)^2\end{aligned}$$

■

5.2 5b

Simulate $n = 30$ and compare jackknife bias and SE with theoretical values $\mathbb{E}[\max X] = \frac{n}{n+1}$, $\text{Var}(\max X) = \frac{n}{(n+2)(n+1)^2}$.

□

```
Q5_n <- 30

Q5_true_theta <- function(n) {
  n / (n + 1)
}

Q5_true_var <- function(n) {
```

```

    n / ((n + 1)^2 * (n + 2))
  }

Q5_simulate_theta_hat <- function(n, B = 10000) {
  theta_hats <- numeric(B)

  for (b in 1:B) {
    sample_data <- runif(n, min = 0, max = 1)
    theta_hats[b] <- max(sample_data)
  }

  mean_theta_hat <- mean(theta_hats)
  var_theta_hat <- var(theta_hats)

  list(mean_theta_hat = mean_theta_hat, var_theta_hat = var_theta_hat)
}

Q5_main <- function() {
  n <- Q5_n
  B <- 10000

  sim_results <- Q5_simulate_theta_hat(n, B)
  true_theta <- Q5_true_theta(n)
  true_var <- Q5_true_var(n)

  theta_error = abs(true_theta - sim_results$mean_theta_hat)
  var_error = abs(true_var - sim_results$var_theta_hat)

  print(paste("Simulated E[theta_hat]: ", round(sim_results$mean_theta_hat, 4)))
  print(paste("True E[theta_hat]: ", round(true_theta, 4)))
  print(paste("Simulated Var(theta_hat): ", round(sim_results$var_theta_hat, 6)))
  print(paste("True Var(theta_hat): ", round(true_var, 6)))
  print("")
  print(paste("E[theta_hat] error: ", round(theta_error, 4)))
  print(paste("Var(theta_hat) error: ", round(var_error, 6)))
}

Q5_main()

```

```

## [1] "Simulated E[theta_hat]: 0.9678"
## [1] "True E[theta_hat]: 0.9677"
## [1] "Simulated Var(theta_hat): 0.000973"
## [1] "True Var(theta_hat): 0.000976"
## [1] ""
## [1] "E[theta_hat] error: 1e-04"
## [1] "Var(theta_hat) error: 2e-06"

```

From the above, we can see that the error is quite low, on the order of 10^{-4} for the expected value and 10^{-6} for the variance. This points towards good convergence of the jackknife bias and variance to the true values. I did not do multiple different trials to determine

■

6 Question 6

6.1 6a

Explain the difference between the sampling distribution F_n , the bootstrap distribution F_n^* , and the population distribution F , and their relationship.

□

So, we have that F is the true population distribution which is not known. This is a fixed but unknown quantity. This represents the true distribution.

F_n however, is the sampled distribution from some observed data. This is estimated by assuming the multiset of observed data of size n , has an equal probability of each data point being selected. This allows us to sample the distribution and build an estimate of F . In fact, if we allow $n \rightarrow \infty$, then $F_n \rightarrow F$. The specific form of F_n is given below where $\mathbf{1}$ represents the indicator function.

$$F_n = \frac{1}{n} \sum_{i=1}^n \mathbf{1}\{X_i \leq x\}$$

The bootstrapped distribution, F_n^* is similar to the empirical distribution but it allows for us to draw samples from F_n . Specifically, to form the F_n^* , we sample a new dataset, X^* , from X where each point can be resampled. We pull n samples all iid from F_n so that $X^* = \{X_1^*, \dots, X_n^*\}$.

The connection of this allows us to say that $F \approx F_n$ and likewise, $F_n \approx F_n^*$ which implies that $F \approx F_n^*$. So F is the true distribution, F_n is a good approximation, and F_n^* is a good and computationally feasible approximation to F_n .

■